

PROJECTS



Getting Started with Delta Lake

This guide helps you quickly explore the [main features](#) of Delta Lake. It provides code snippets that show how to read from and write to Delta tables from interactive, batch, and streaming queries. It also demonstrates table updates and time travel.

Set up Apache Spark with Delta Lake

Follow these instructions to set up Delta Lake with Spark. You can run the steps in this guide on your local machine in the following two ways:

1. Run interactively: Start the Spark shell (Scala or Python) with Delta Lake and run the code snippets interactively in the shell.
2. Run as a project: Set up a Maven or SBT project (Scala or Java) with Delta Lake, copy the code snippets into a source file, and run the project. Alternatively, you can use the [examples provided in the GitHub repository](#).

Set up interactive shell

To use Delta Lake interactively within the Spark Scala or Python shell, you need a local installation of Apache Spark. Depending on whether you want to use Python or Scala, you can set up either PySpark or the Spark shell, respectively. For all the instructions below make sure you install the correct version of Spark or PySpark that is compatible with Delta Lake 2.1.0. See the release compatibility matrix for details.

PySpark shell

1. Install the PySpark version that is compatible with the Delta Lake version by running the following:

```
pip install pyspark==<compatible-spark-version>
```

2. Run PySpark with the Delta Lake package and additional configurations:

```
pyspark --packages io.delta:delta-core_2.12:2.1.0 \
  --conf "spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension" \
  --conf "spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog"
```



Spark Scala shell

Download the compatible version of Apache Spark by following instructions from [Downloading Spark](#), either using `pip` or by downloading and extracting the archive and running `spark-shell` in the extracted directory.

```
bin/spark-shell --packages io.delta:delta-core_2.12:2.1.0 \
  --conf "spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension" \
  --conf "spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog"
```



Set up project

If you want to build a project using Delta Lake binaries from Maven Central Repository, you can use the following Maven coordinates.

Maven

You include Delta Lake in your Maven project by adding it as a dependency in your POM file. Delta Lake compiled with Scala 2.12.

```
<dependency>
  <groupId>io.delta</groupId>
  <artifactId>delta-core_2.12</artifactId>
  <version>2.1.0</version>
</dependency>
```

SBT

You include Delta Lake in your SBT project by adding the following line to your `build.sbt` file:

```
libraryDependencies += "io.delta" %% "delta-core" % "2.1.0"
```

Python

To set up a Python project (for example, for unit testing), you can install Delta Lake using `pip install delta-spark` and then configure the `SparkSession` with the `configure_spark_with_delta_pip()` utility function in Delta Lake:

```
from delta import *

builder = pyspark.sql.Session.builder.appName("MyApp") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkS
```

```
.config("spark.sql.catalog.spark_catalog", "org.apache.spark
```

```
SQLCatalog$.MODULE$.getOrCreate())
```

Create a table

To create a Delta table, write a DataFrame out in the delta format. You can use existing Spark SQL code and change the format from parquet, csv, json, and so on, to delta.

```
data = spark.range(0, 5)
data.write.format("delta").save("/tmp/delta-table")
```

These operations create a new Delta table using the schema that was *inferred* from your DataFrame. For the full set of options available when you create a new Delta table, see the [Create a table](#) and [Write to a table](#) documentation articles.

Note

This guide uses local paths for Delta table locations. For configuring HDFS or cloud storage for Delta tables, see the [Storage configuration](#) Documentation article.

Read data

You read data in your Delta table by specifying the path to the files

```
"/tmp/delta-table":
```

```
df = spark.read.format("delta").load("/tmp/delta-table")
df.show()
```

Update table data

Delta Lake supports several operations to modify tables using standard DataFrame APIs. This example runs a batch job to overwrite the data in the table:

Overwrite

```
data = spark.range(5, 10)
data.write.format("delta").mode("overwrite").save("/tmp/delta-table")
```



If you read this table again, you should see only the values 5-9 you have added because you overwrote the previous data.

Conditional update without overwrite

Delta Lake provides programmatic APIs to conditional update, delete, and merge (upsert) data into tables. Here are a few examples:

```
from delta.tables import *
from pyspark.sql.functions import *

deltaTable = DeltaTable.forPath(spark, "/tmp/delta-table")

# Update every even value by adding 100 to it
deltaTable.update(
    condition = expr("id % 2 == 0"),
    set = { "id": expr("id + 100") })

# Delete every even value
deltaTable.delete(condition = expr("id % 2 == 0"))

# Upsert (merge) new data
newData = spark.range(0, 20)

deltaTable.alias("oldData") \
    .merge(
```

```
newData.alias("newData"),
"oldData.id = newData.id") \
.whenMatchedUpdate(set = { "id": col("newData.id") }) \
.whenNotMatchedInsert(values = { "id": col("newData.id") })
.execute()
```

```
df.write.format("delta").overwrite().saveAsTable("delta-table")
```

You should see that some of the existing rows have been updated and new rows have been inserted.

For more information on these operations, see the [Table deletes, updates, and merges](#) documentation article.

Read older versions of data using time travel

You can query previous snapshots of your Delta table by using time travel. If you want to access the data that you overwrote, you can query a snapshot of the table before you overwrote the first set of data using the `versionAsOf` option.

```
df = spark.read.format("delta") \
.option("versionAsOf", 0) \
.load("/tmp/delta-table")

df.show()
```

You should see the first set of data, from before you overwrote it. Time travel takes advantage of the power of the Delta Lake transaction log to access data that is no longer in the table. Removing the version 0 option (or specifying version 1) would let you see the newer data again. For more information, see the [Query an older snapshot of a table \(time travel\)](#) documentation article.

Write a stream of data to a table

You can also write to a Delta table using Structured Streaming. The Delta Lake transaction log guarantees exactly-once processing, even when there are other streams or batch queries running concurrently against the table. By default, streams run in append mode, which adds new records to the table:

```
streamingDf = spark.readStream.format("rate").load()

stream = streamingDf \
    .selectExpr("value as id") \
    .writeStream.format("delta") \
    .option("checkpointLocation", "/tmp/checkpoint") \
    .start("/tmp/delta-table")
```

While the stream is running, you can read the table using the earlier commands.

Note

If you're running this in a shell, you may see the streaming task progress, which make it hard to type commands in that shell. It may be useful to start another shell in a new terminal for querying the table.

You can stop the stream by running `stream.stop()` in the same terminal that started the stream.

For more information about Delta Lake integration with Structured Streaming, see the [Table streaming reads and writes](#) documentation article.

Read a stream of changes from a table

While the stream is writing to the Delta table, you can also read from that table as streaming source. For example, you can start another streaming query that prints all the changes made to the Delta table. You can specify which version Structured Streaming should start from by providing the `startingVersion` or `startingTimestamp` option to get changes from that point onwards. See the [Structured Streaming](#) documentation article for details.

```
stream2 = spark.readStream.format("delta") \
    .load("/tmp/delta-table") \
    .writeStream.format("console") \
    .start()
```

Next Steps

Here are some further resources for learning more about Delta Lake:

- [Delta FAQ](#) for answers to common questions about Delta Lake
- [Tutorials](#) for video tutorials about Delta Lake
- [Documentation—Batch Reading and Writing](#) for detailed documentation on table batch reads and writes
- [Examples](#) for Delta Lake usage examples

DELTA LAKE

Sharing

Integrations

Roadmap

Blogs

LEARN

Getting Started

Blogs

Tutorials

Videos

Case Studies

COMMUNITY

Community

Getting Help

Contributing to Delta

SOCIAL CHANNELS

StackOverflow

GitHub

Twitter

Slack

LinkedIn

Copyright © 2024 Delta Lake, a series of LF Projects, LLC. For web site terms of use, trademark policy and other project policies please see <https://lfprojects.org>.

